

A Minimalist Datatype-Generic Putback-Based Bidirectional Programming Language

Hsiang-Shang Ko², Tao Zan¹, and Zhenjiang Hu^{1,2}

¹ SOKENDAI (The Graduate University for Advanced Studies), Japan

² National Institute of Informatics, Japan

1 Introduction

[1]

2 Decidable partiality

The bidirectional transformations discussed in this paper are “decidably partial”. By “decidable partiality” we mean that the transformations are actually total functions which wrap their results in the standard `Maybe` datatype:

```
data Maybe (A : Set) : Set where
  just    : A → Maybe A
  nothing : Maybe A
```

Explanation of
AGDA datatype definition

The result of a piece of successful computation is wrapped in the `just` constructor, while failed computation is represented by `nothing`. The behaviour of decidably partial functions is different from the usual partial functions as formalised in terms of complete partial orders (see, e.g., [2]): In Section 4 we will need to define programs that produce some result or fail respectively when a sub-program fails or produces some result, but this is not possible in the setting of complete partial orders (since such functions violate monotonicity, a necessary condition for computability).

We can choose to write our decidably partial bidirectional transformations directly as `Maybe`-programs, but when proving their well-behavedness, we will need to analyse or establish how a piece of decidably partial computation succeeds or fails, and this task would be much easier if the computation steps leading to success or failure are reified as data. The reification starts from a deep embedding of the combinators that we will use for writing decidably partial programs:

AGDA naming rules

```
data Par : Set → Set1 where
  return      : { A : Set } → A → Par A
  _>>_       : { A B : Set } → Par A → (A → Par B) → Par B
  fail        : { A : Set } → Par A
  catch       : { A B : Set } → Par A → (A → Par B) → Par B → Par B
  assert_then_ : { A B : Set } → Bool → Par A → Par A
```

Informally: A value of type `Par A` represents a piece of decidable partial computation that successfully produces a value of type `A` or fails. The first two constructors `return` and `_>>=` represent the usual monadic operators [3,4], and `fail` indicates failed computation. We will also need error handling, which is represented by the `catch` constructor. The behaviour of our `catch` is slightly different from the usual one: execution of `catch mx f my` binds the result of `mx` to `f` if `mx` computes successfully, or goes to execution of `my` if `mx` fails to compute. Also we will frequently need to ensure that a `Bool` value is `true` before continuing with the rest of a program, so the constructor `assert_then_` is included for convenience — execution of `assert b then p` becomes execution of `p` when `b` is `true`, or fails otherwise. Formally, the meaning of the constructors of `Par` is given by the following interpreter to `Maybe`:

```

runPar : { A : Set } → Par A → Maybe A
runPar (return x) = just x
runPar (mx >>= f) with runPar mx
runPar (mx >>= f) | just x   = runPar (f x)
runPar (mx >>= f) | nothing = nothing
runPar fail = nothing
runPar (catch mx f my) with runPar mx
runPar (catch mx f my) | just x   = runPar (f x)
runPar (catch mx f my) | nothing = runPar my
runPar (assert true then mx) = runPar mx
runPar (assert false then mx) = nothing

```

Equality type

We say that $mx : \text{Par } A$ computes to $x : A$ exactly when $\text{runPar } mx \equiv \text{just } x$, and that mx fails to compute exactly when $\text{runPar } mx \equiv \text{nothing}$. These two equalities are what we aim to reify as data next.

Our reification of decidable partial computation as the datatype `Par` and AGDA’s support of full dependent types allow us to compute for any $mx : \text{Par } A$ and $x : A$ a type $mx \mapsto x$ whose inhabitants explain how mx computes to x . The definition of this family of types is shown in Figure 1: `return x` computes to x' when x is exactly x' , and $mx \gg= f$ computes to y when there exists $x : A$ such that mx computes to x and $f x$ computes to y , and so on. Here our use of the term “computes to” is justified, since we can prove that

$$mx \mapsto x \quad \text{and} \quad \text{runPar } mx \equiv \text{just } x$$

are equivalent, i.e., there are functions converting between the two types. In other words, we can prove $mx \mapsto x$ if and only if $\text{runPar } mx \equiv \text{just } x$, meaning that our reification of $\text{runPar } mx \equiv \text{just } x$ as $mx \mapsto x$ is accurate. In the clause for `catch mx f my`, a possible situation for the computation to succeed is when mx fails but my succeeds subsequently. Thus the ways a piece of computation fails have to be reified as a family of types too. This family of types is defined as $\neg\mapsto$ (mutually with $\neg\mapsto$), also in Figure 1. Similarly, we can prove that

$$mx \neg\mapsto \quad \text{and} \quad \text{runPar } mx \equiv \text{nothing}$$

mutual

$$\begin{aligned}
& _ \mapsto _ : \{A : \text{Set}\} \rightarrow \text{Par } A \rightarrow A \rightarrow \text{Set}_1 \\
& \text{return } x \quad \mapsto x' = x \equiv x' \\
& mx \ggg f \quad \mapsto y = (x : A) \times (mx \mapsto x) \times (f x \mapsto y) \\
& \text{fail} \quad \mapsto x = \perp \\
& \text{catch } mx \text{ f } my \quad \mapsto z = ((x : A) \times (mx \mapsto x) \times (f x \mapsto z)) \uplus \\
& \quad \quad \quad ((mx \not\mapsto) \times (my \mapsto z)) \\
& \text{assert } b \text{ then } mx \mapsto x = (b \equiv \text{true}) \times (mx \mapsto x) \\
& _ \not\mapsto _ : \{A : \text{Set}\} \rightarrow \text{Par } A \rightarrow \text{Set}_1 \\
& \text{return } x \quad \not\mapsto = \perp \\
& mx \ggg f \quad \not\mapsto = (mx \not\mapsto) \uplus ((x : A) \times (mx \mapsto x) \times (f x \not\mapsto)) \\
& \text{fail} \quad \not\mapsto = \top \\
& \text{catch } mx \text{ f } my \quad \not\mapsto = ((mx \not\mapsto) \times (my \not\mapsto)) \uplus \\
& \quad \quad \quad ((x : A) \times (mx \mapsto x) \times (f x \not\mapsto)) \\
& \text{assert } b \text{ then } mx \not\mapsto = (b \equiv \text{false}) \uplus ((b \equiv \text{true}) \times (mx \not\mapsto))
\end{aligned}$$

Fig. 1. Definitions of $_ \mapsto _$ and $_ \not\mapsto _$.

are equivalent, showing that the reification of $\text{runPar } mx \equiv \text{nothing}$ as $mx \not\mapsto$ is, again, accurate. We are thus entitled to use $_ \mapsto _$ and $_ \not\mapsto _$ for stating various properties involving success or failure of decidable partial computation, in particular well-behavedness of bidirectional transformations in the next section.

3 Putback-based bidirectional transformations

record $\text{Lens } (S \ V : \text{Set}) : \text{Set}_1$ **where**
field

$$\begin{aligned}
& \text{put} : S \rightarrow V \rightarrow \text{Par } S \\
& \text{get} : S \rightarrow \text{Par } V \\
& \text{PutGet} : \{s \ s' : S\} \{v : V\} \rightarrow (\text{put } s \ v \mapsto s') \rightarrow (\text{get } s' \mapsto v) \\
& \text{GetPut} : \{s : S\} \{v : V\} \rightarrow (\text{get } s \mapsto v) \rightarrow (\text{put } s \ v \mapsto s)
\end{aligned}$$

$$\begin{aligned}
& \text{get } s \mapsto v \\
& \Rightarrow \{ \text{GetPut for } \text{get} \} \\
& \text{put } s \ v \mapsto s \\
& \Rightarrow \{ \text{PutGet for } \text{get}' \} \\
& \text{get}' s \mapsto v
\end{aligned}$$

4 BiGUL and its lens semantics

5 Reduction of common features

6 Discussion

Haskell implementation

Towards a dependently typed version

Presentation in AGDA

References

1. J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.
2. Jeremy Gibbons. Calculating functional programs. In *Algebraic and Coalgebraic Methods in the Mathematics of Program Construction*, number 2297 in Lecture Notes in Computer Science, chapter 5, pages 149–201. Springer-Verlag, 2002.
3. Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.
4. Philip Wadler. The essence of functional programming. In *Principles of Programming Languages*, POPL’92, pages 1–14. ACM, 1992.